

**Compte rendu 5 semaine de stage
en mise en place d'un réseaux
ad-hoc et service de journalisation**

Sommaire

- I. Présentation de l'entreprise**
- II. Objectif du stage**
- III. Outils et choix matériels**
- IV. Travail réalisé**
 - A. Choix des OS**
 - B. Construction du script**
 - 1. Script OLSR (gamma.sh)**
 - 1. Mise en service
 - 2. Étape [1/18]
 - 3. Étape [2/18]
 - 4. Étape [3/18]
 - 5. Étape [4/18]
 - 6. Étape [5/18]
 - 7. Étape [6/18]
 - 8. Étape [7/18]
 - 9. Étape [8/18]
 - 10. Étape [9/18]
 - 11. Étape [10/18]
 - 12. Étape [11/18]
 - 13. Étape [12/18]
 - 14. Étape [13/18]
 - 15. Étape [14/18]
 - 16. Étape [15/18]
 - 17. Étape [16/18]
 - 18. Étape [17/18]
 - 19. Étape [18/18]
 - C. Topologie de teste**
 - 1. Bureau
 - 2. Sous sol
 - 3. BATMAN-adv
 - D. Commande utile**
 - E. Lien utile**
- V. Conclusion**
- VI. Annexe**

I-Présentation de l'ONERA Toulouse

Introduction

Dans le cadre de mon stage, j'ai intégré le centre de Toulouse de l'ONERA (Office national d'études et de recherches aérospatiales), principal organisme public français de recherche dans le domaine aéronautique, spatial et de défense. Il contribue au développement scientifique et technologique du secteur et accompagne les industriels dans leurs projets d'innovation.

Présentation générale de l'ONERA

Créé en 1946, l'ONERA est un établissement public à caractère industriel et commercial (EPIC) placé sous la tutelle du ministère des Armées. Il mène des recherches amont et appliquées pour la conception et l'amélioration des aéronefs, satellites et systèmes de défense. Ses missions principales sont de développer les connaissances scientifiques, concevoir et exploiter des moyens expérimentaux, accompagner les industriels et former des doctorants et ingénieurs.

Le centre ONERA de Toulouse

Situé sur le campus scientifique de Rangueil, le centre de Toulouse se trouve au cœur d'un environnement marqué par l'industrie aéronautique et spatiale, favorisant les collaborations académiques et industrielles. Les équipes travaillent sur le traitement de l'information et les systèmes embarqués, l'électromagnétisme et les technologies radar, l'optique, la physique de l'environnement spatial et la modélisation multiphysique.

Activités et rôle

L'ONERA Toulouse conduit des recherches fondamentales et appliquées, élabore des modèles théoriques, conçoit des outils numériques et réalise des expérimentations pour améliorer la compréhension et la performance des systèmes aérospatiaux. Le centre participe à des projets nationaux et européens et accueille des doctorants et stagiaires, renforçant le lien entre recherche et formation.

II- Objectif du stage

L'objectif principal de mon stage a été de développer et tester des applications fonctionnant sur des **Raspberry Pi Zero 2 W** au sein d'un réseau mesh ad-hoc. Ce réseau repose sur des nœuds autonomes communiquant entre eux et coordonnés par un **nœud maître** basé sur une Raspberry Pi 3. L'expérimentation visait à étudier à la fois la gestion du trafic et la génération de données au sein du réseau.

Le stage s'est articulé autour de deux applications distinctes mais complémentaires. La première application a été conçue pour **surveiller le réseau**. Elle collecte et affiche des informations sur l'état des nœuds, la disponibilité des liaisons et le trafic passant par chaque nœud. Ces données permettent de visualiser le fonctionnement du réseau en temps réel et d'identifier d'éventuels goulots d'étranglement ou pertes de paquets. Cette application a été développée avec un souci de légèreté et de compatibilité avec les ressources limitées des Raspberry Pi Zero 2 W.

La seconde application a pour rôle de **générer du trafic applicatif** afin de tester le réseau. Elle envoie différents types de données, allant de simples messages texte à des flux vidéo ou des pings entre nœuds. L'objectif est de simuler des scénarios d'utilisation réalistes et de mesurer les performances du réseau en conditions variées. La combinaison des deux applications permet d'analyser la robustesse, la réactivité et l'efficacité du réseau mesh.

Le choix des Raspberry Pi Zero 2 W s'est avéré pertinent pour ce stage, car ces cartes offrent un compromis entre puissance de calcul et faible consommation, tout en permettant de simuler un réseau de nœuds relativement dense. Le nœud maître sur Raspberry Pi 3 centralise la collecte de données et la coordination du réseau, garantissant une expérimentation fiable et reproductible.

Au cours du stage, le développement a inclus la programmation des applications, la configuration des nœuds et la mise en place d'un protocole de communication simple mais efficace. Les tests ont permis de valider le bon fonctionnement du réseau, de mesurer la latence et le débit des communications, et de générer des rapports détaillés sur le comportement du réseau selon différentes charges applicatives.

Cette expérience m'a permis de mettre en pratique des compétences en **programmation embarquée**, en **réseaux ad-hoc** et en **analyse de performance**, tout en travaillant dans un environnement proche des contraintes industrielles. Le stage a également renforcé ma compréhension des défis liés aux systèmes distribués et à la communication entre multiples nœuds dans un réseau dynamique.

III-Outils et choix matériels

Pour réaliser mon expérimentation sur un réseau mesh ad-hoc, j'ai choisi d'utiliser des **Raspberry Pi Zero W2 (voir annexe 1)** comme nœuds du réseau et une **Raspberry Pi 3 (voir annexe 2)** comme nœud maître. Ce choix a été motivé par plusieurs critères : accessibilité, flexibilité et faible consommation énergétique. Les Raspberry Pi offrent une interface simple pour développer et tester des protocoles de communication, tout en restant proches des contraintes matérielles que l'on pourrait rencontrer sur des systèmes embarqués réels.

D'autres solutions matérielles, comme les cartes Arduino ou les modules ESP32, ont été envisagées. Cependant, un ESP32 ne peut pas faire fonctionner nativement un véritable réseau ad hoc 802.11 IBSS car sa pile Wi-Fi, fournie par ESP-IDF, ne prend en charge que les modes station et softAP et n'expose pas le contrôle MAC de bas niveau nécessaire au routage ad hoc conforme aux normes. Cependant, il peut toujours former des réseaux dynamiques multi-sauts en utilisant des cadres Espressif comme ESP-MESH ou des communications entre pairs sans connexion via ESP-NOW. Dans ces configurations, les développeurs peuvent mettre en œuvre une logique de routage ou de transfert personnalisée au niveau de l'application ou de la couche maillée, ce qui permet une communication multi-nœuds à autorégénération, mais avec des limites de ressources plus strictes et moins de souplesse de protocole que les véritables mises en œuvre de MANET IBSS. Pour avoir accès à ces fonctions, nous avons besoin d'un environnement Linux.

L'utilisation de Raspberry Pi a permis de disposer d'un environnement complet, capable d'exécuter le logiciel de gestion du réseau et de simuler des scénarios réalistes, tout en restant modulaire pour l'ajout ou le retrait de nœuds. Ainsi, ce choix permet de justifier l'expérimentation sur un réseau fonctionnel et reproductible, tout en offrant un cadre proche des applications industrielles pour lesquelles la scalabilité et la robustesse des communications sont des facteurs clés.

IV-Travaux réalisés

A-Choix des OS

Le choix du système d'exploitation constitue une étape clé pour le développement et le déploiement des applications sur un réseau de Raspberry Pi. Dans le cadre de mon stage, j'ai étudié plusieurs distributions afin de sélectionner celle qui offrirait la meilleure stabilité, compatibilité matérielle et facilité de configuration pour les nœuds du réseau et le nœud maître.

Plusieurs options ont été évaluées. La distribution **Trixie** a été envisagée en raison de ses fonctionnalités modernes et de son support pour les dernières versions des logiciels. Cependant, elle a été écartée car il s'agit d'une licence récente, encore peu utilisée dans les communautés techniques. La documentation disponible reste limitée, ce qui aurait pu poser des difficultés pour l'installation, la configuration et le débogage des applications développées pour le stage. **Non retenu.**

La distribution **Ubuntu Server** a également été considérée. Elle bénéficie d'une large base d'utilisateurs et d'un suivi actif. Toutefois, son utilisation sur les Raspberry Pi a été rendue compliquée par des problèmes liés à l'outil Imager, entraînant des erreurs d'installation et des incompatibilités avec certains périphériques. Ces difficultés auraient allongé le temps de mise en place et introduit des risques pour la stabilité des expérimentations. **Non retenu.**

Finalement, le choix s'est porté sur la distribution **Debian Bookworm**, adaptée aux Raspberry Pi et relativement stable. Bien que cette licence soit plus ancienne, elle présente plusieurs avantages décisifs : la documentation est abondante, les outils de paramétrage sont simples à utiliser et la compatibilité avec le matériel Raspberry Pi est éprouvée. Cette distribution permet d'installer rapidement les bibliothèques nécessaires et de déployer les scripts expérimentaux sans rencontrer de blocages techniques majeurs. **Solution retenue !**

L'utilisation de Debian Bookworm a également facilité l'intégration avec le nœud maître basé sur une Raspberry Pi 3, garantissant une uniformité dans l'environnement logiciel et une meilleure gestion des communications au sein du réseau mesh. Ce choix contribue à assurer la reproductibilité des tests, la fiabilité des mesures de trafic et la simplicité des mises à jour logicielles au cours du stage.

En résumé, Debian Bookworm a été retenu pour sa stabilité, sa compatibilité matérielle, sa documentation complète et sa facilité de configuration, répondant ainsi aux besoins spécifiques du projet. Ce choix s'inscrit dans une logique de minimisation des risques techniques et de maximisation de l'efficacité lors de la mise en œuvre des applications sur les nœuds du réseau.

B-Construction d'un script

1-Script OLSR (gamma.sh)

Dans cette section nous allons décrire la structure des codes rédigés sous un format de sommaire.

1. Lancement
2. Téléchargement olsr
3. Log
4. Mise en place de service

Avant le lancement des étapes

Mise en place d'un système d'alerte en cas de mauvais lancement du script,

Mise en place des journaux et système interactif de log.

Étape [1/18]

Mise en place du fichier qui permet de traverser le proxy,

Mise en place de la date,

Installation des apt.

Étape [2/18]

Désactivation de service.

Étape [3/18]

Activation de service.

Étape [4/18]

Mise en place du serveur de synchronisation de date et d'heure .

Étape [5/18]

Installation et décompression OLSR

Étape [6/18]

Tentative de configuration OLSR

Étape [7/18]

Exclusion de Wlan0

Étape [8/18]

Passage de Wlan0 en IBSS

Étape [9/18]

Passage en IBSS et log

Étape [10/18]

Script de vérification et contrainte IBSS

Étape [11/18]

Ping broadcast

Étape [12/18]

Forçage du lancement de OLSR

Étape [13/18]

Check OLSR ok au boot

Étape [14/18]

Script .json situation de teste

Étape [15/18]

Script .json maître

Étape [16/18]

Script .json 600s

Étape [17/18]

Service de reset de l'ip après le boot

Étape [18/18]

Log et fin du script

Mise en service

La mise en service du script se déroule en **3 étapes** bien délimitées. La première partie sert de gardien afin de vérifier que vous soyez bien en sudo et que vous avez bien pensé à joindre le numéro qui qualifie l'ip du script. Il y a ensuite la création d'un fichier de log dans **/var/log/logmaj.txt** qui enregistre toutes les info qui seront remontées dans le script et ensuite, il y a l'ajout d'un **entête** pour désigner le début du script.

Étape [1/18]

Mise en place d'une condition qui **supprime** le fichier **95proxy** si il existe déjà, puis, il le **copie** dans le fichier **/etc/apt/apt.conf.d/** ensuite **mise à jour manuel de la date** afin de ne pas avoir d'apt qui rate, puis installation de tous les **apt nécessaires** à la mise en place du script.

Étape [2/18]

Désactivation du service **NetworkManager** et du service **wpa_supplicant** qui empêche le passage en mode IBSS

Étape [3/18]

Log du lancement de **systemd-networkd** et **lancement** de ce service.

Étape [4/18]

Mise en place de deux boucles, une première, qui va s'occuper d'identifier le nœud **maître** (10.0.0.1) et installation d'un **serveur chrony** dessus afin de distribuer l'heure et la date à toutes les machines clients qui viennent chercher ces informations directement dessus. Le deuxième algorithme vient chercher la **date** sur toutes les ip du réseaux 10.0.0.x/16 sur le nœud maitre (le 10.0.0.1) afin de mettre toutes les machine au là.

Étape [5/18]

Déplacement de **OONF** et don des droit superuser sur le dossier,
Mise en place de la **liste cmake**,
Lancement des installations,
Ajout d'une boucle de vérification à la fin afin de certifier que les **librairies** sont bien installées.

Étape [6/18]

Tentative de **configuration de olsr**, cependant, il y as un problème majeur, lors la configuration, quand je souhaite demander à OLSR d'utiliser des

paramètres de personnalisation, il cherché à piocher le nom de l'interface ou il souhaite modifier les paramètres de manière **dynamique** et, il échoue à chaque fois, cela rendait donc les **modifications impossible** et donc sont utilisation par la même occasion.

Étape [7/18]

Dans cette étape, on essaye de faire **sortir Wlan0** des griffes du méchant système qui essaye continuellement de nous remettre Wlan0 en interface de fonctionnement simple. C'est à dire qu'il cherche à sortir Wlan0 du **mode IBSS** que l'on essaye de lui imposer.

Étape [8/18]

On tente de mettre les **paramètres d'exclusion** des services problématique en appliquant directement dans le fichier qui s'occupe de gérer la configuration réseaux ces paramètres.

Étape [9/18]

Tentative de remise en place du **mode IBSS** (car en général la première fois cela avait sauté) ajoute de **log à chaque étape** afin de prendre le recule nécessaire et d'identifier les étapes qui pose problème afin d'aligner l'étape suivante (Je conseil de garder cette étape simplement pour éviter que l'interface ne soit plus en mode IBSS au redémarrage)

Étape [10/18]

Mise en place d'un autre fichier qui explique au système que l'interface Wlan0 est maintenant en IBSS et affiche l'état du daemon directement depuis la zone d'exécution après sont redémarrage.

Étape [11/18]

Cette étape sert à mettre en place un script qui vas se lancer au démarrage et envoyer des **requêtes ICMP** en broadcast un peu n'importe ou sur le réseaux **10.0.0.x** afin de créer la liste des routes disponibles dans le réseaux et de pouvoir les afficher avec **ip neigh show dev wlan0**.

Étape [12/18]

Tentative de configuration d'un service de **OLSR** (ça marche pas) et tentative de **démarrage du daemon** (c'est toujours un échec)

Étape [13/18]

Script qui lance une boucle qui va **chercher pendant 300s** si OLSR retourne Active quand on fait un **systemctl status OLSRv2.service**. Si le service ne tourne pas alors le **script crash**. Il y a ensuite une vérification que le service **Telnet** et le service de **Fowarding** tourne sinon crash du script.

Étape [14/18]

Mise en place d'un script un peu différent des autres mais qui en finalité ressemble énormément à celui de l'**étape 16** juste pour le nœud maître.

Étape [15/18]

Mise en place d'un script particulier, il fonctionne uniquement sur le **nœud maître** et permet à celui-ci de se connecter en **ssh** sur toute les machines du réseaux 10.0.0.x et exécute la commande qui permet de la lancé le **script clients**, permet en finalité de ce connecter en ssh sur les machines sur lequel elle as lancé le scripte client et de partir **recupérer le script** et d'en faire une **copie** sur lui.

Étape [16/18]

Script client qui lance une séquence durant laquelle elle envoie des **requêtes ICMP (ping)** dans le **réseaux 10.0.0.0** et récolte les rendus de chaque ping dans des tableaux **.json**, ceci et fait pendant **600s** à partir de départ de l'ordre donnée.

Étape [17/18]

Mise en place d'un dernier script qui **remplace toute les informations** d'ip mise sur **Wlan0** pour qu' il appartienne bien au **réseaux 10.0.0.x** .

Étape [18/18]

Dernière étape, mise en Log de fin de script et de redémarrage pour mettre en place tous les services.

C- Mise en place de topologie teste

Mise en place de topologie afin de tester dans la nature les cartes et la robustesse du script.

Topologie 1 : Bureau

Mise en place de cette topologie (**Voir annexe 1**) afin de prendre des valeurs de référence et vérifier que les puces soient bien logées dans les fichiers .json .

Topologie 2 : Sous sol

Mise en place de cette topologie (**Voir annexe 2**) afin de visualiser une formation en bus et la solidité des routes.

Topologie 3 : BATMAN-adv

Mise en place de cette topologie (**Voir annexe 3**) afin de visualiser une formation en ligne droite avec une cible mouvante (Entourer en noir) qui se déplace dans le périmètre des autres puces, afin de tester la souplesse des routes.

D-Commande utile

Pour voir l'état des routes: *ip neigh show dev wlan0*

Pour voir l'état d'un service: *systemctl status olsrv2.service*

Pour lancer le service de journalisation pendant 10 min: *sudo mesh-run-icmp*

Pour voir les commandes utilisables dans cette version de Olsr:
/usr/sbin/olsrd2_dynamic --schema

Uniquement pour batman : *mesh-info*

E-Lien utile

Petit résumé des sources utilisées lors de mes recherches ou, utile pour la suite,

Site de doc de BATMAN : <https://www.open-mesh.org/projects/batman-adv/wiki/Wiki>

Site de doc OLSRV2 : <https://openwrt.org/docs/guide-user/start>

Site de téléchargement: <https://www.raspberrypi.com/software/operating-systems/>

Site de recherche le plus utilisé : <https://chatgpt.com/>

Site de recherche le plus pertinent : <https://claude.ai>

Site installation OONF 'stable' 0.15.1 :

<https://github.com/OLSR/OONF/archive/refs/tags/v0.15.1.tar.gz>

V- Expérience de teste

A-(OLSR)Bureau

Pour effectuer ce test, j'ai simplement installé le script gamma.sh sur toutes les puces que j'avais à ma disposition.

J'ai ensuite lancé le script sudo mesh-run-icmp

Je laisse le script tourné puis je regarde que tous les fichiers sont bien copiés sur mon nœud maître. Une fois cela effectué je jette un coup d'œil au résultat et dans les faits, tous les nœuds on répond et on tous pu communiquer les un avec les autres.

Interprétation :

Passage d'un cap dans l'évolution de mon stage, passage du développement au test.

B-(OLSR)Sous sol

Cette topologie a été pensée par mon génialissime superviseur de stage, Hugo, cette topologie permet de tester un bus à plusieurs sorties soit un topologie qui n'est pas anodine car non réalisable avec une couche physique différente. Pour tester cette configuration nous avons donc mis les Raspberry dans la configuration de l'annexe 2 puis avant essayer de lancer le script .json qui as pour but d'envoyer des requests ICMP ainsi que de créer et maintenir les routes entres les noeud, cependant, lors de cette expérience, nous avons pu voir que les pings n'arrivé pas à N+1 (noeud + 1 saut), il fallait une connexion direct pour faire transiter des information

Interprétation :

Problème de configuration OLSR. Cette topologie à permis de mettre en évidence que le script possède des failles de LVL 99.

C-BATMAN-adv (Sous sol)

Pour cette expérience nous avons repris la topologie sous sol afin de pouvoir tester la robustesse des routes, ceci fait nous avons pu voir que ce script était belle et bien fonctionné mais non adapté nécessairement pour faire transiter des information sur le réseau car une fois que la mise en place fut terminée, nous avons pu voir que la latence entre les noeud devient rapidement importante et ne supporter pas un trafic important.

Interprétation :

Réussite au niveau de la mise en place du système natif BATMAN-adv de bookworm. Cependant, problème non réglé au niveau des logs, car l'affichage du nombre de saut effectué par les packet n'est pas affiché.

D- BATMAN-adv (Mix)

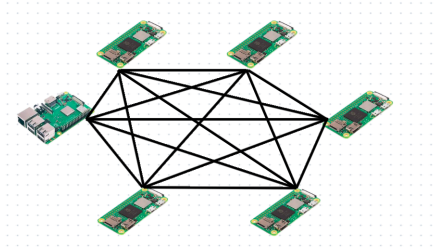
Pour cette topologie, c'est une mise en place primaire qui sert simplement à faire des tests rapide à l'abri de la pluie.

Interprétation :

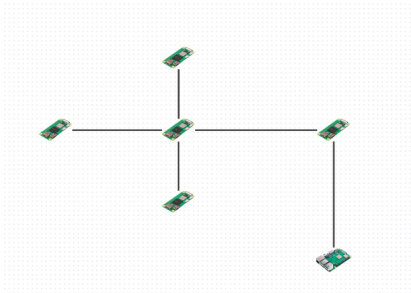
Mise en place de cette topologie pour mettre en place des tests comme le correctif du .json.

VI-Annexe

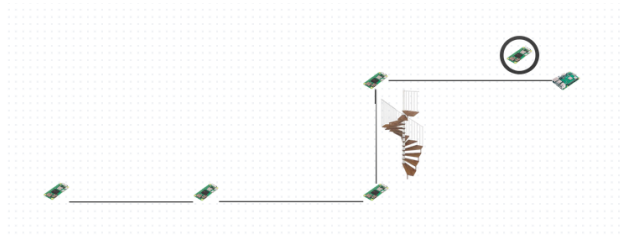
Annexe 1 : Topologie Bureau



Annexe 2 : Topologie Sous sol



Annexe 3 : Topologie Mix



Annexe 4 : [Script.sh](#)

```
#!/bin/bash
#####
# Raspberry Pi Zero W – Mesh BATMAN-adv (Bookworm)
# Usage: sudo bash setup-mesh-batman.sh [BOARD_ID]
#####

### --- Sécurité ---
if [ "$EUID" -ne 0 ]; then
    echo "Lancer ce script avec sudo"
    exit 1
fi

if [ -z "$1" ]; then
    echo "Usage: sudo bash setup-mesh-batman.sh [BOARD_ID]"
    exit 1
fi

BOARD_ID="$1"
IP="10.0.0.$BOARD_ID"
ESSID="mesh-test"
CHANNEL_FREQ="2437" # canal 6

echo "====="
echo " Mesh BATMAN-adv – Node $BOARD_ID ($IP)"
echo "====="

#####
# Logging & status
#####
LOGFILE="/var/log/logmaj.txt"

mkdir -p /var/log
touch "$LOGFILE"
chmod 644 "$LOGFILE"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOGFILE"
}

step_start() {
    STEP_NAME="$1"
    STEP_ID="$2"
    log "[STEP $STEP_ID] $STEP_NAME : START"
}

step_ok() {
```

```
log "[STEP $STEP_ID] $STEP_NAME : OK"
}
```

```
step_fail() {
log "[STEP $STEP_ID] $STEP_NAME : FAIL"
log "Arrêt du script (erreur bloquante)"
exit 1
}
```

```
run() {
"$@" >>"$LOGFILE" 2>&1 || step_fail
}
```

```
#####
# Systemd logging helpers
#####
log_systemd_status() {
SERVICE="$1"
{
echo "----- systemd status : $SERVICE -----"
systemctl is-enabled "$SERVICE" 2>&1 || true
systemctl is-active "$SERVICE" 2>&1 || true
systemctl status "$SERVICE" --no-pager -l 2>&1
echo "-----"
} >>"$LOGFILE"
}
```

```
log "==== DÉMARRAGE INSTALLATION MESH BATMAN-adv ====="
log "BOARD_ID=$BOARD_ID"
```

```
#####
# [1/16] Paquets requis
#####
echo "[1/16] Installation des dépendances..."
step_start "[1/16] Installation des dépendances..." "1/16"
```

```
if [ -f /etc/apt/apt.conf.d/95proxy ]; then
rm /etc/apt/apt.conf.d/95proxy
fi
```

```
if [ -f 95proxy ]; then
cp 95proxy /etc/apt/apt.conf.d/
fi
```

```
date -s "18 FEB 2026 16:45:00"
```

```
apt update
```



```
apt install -y \
iw iproute2 netcat-openbsd \
batctl \
bridge-utils \
systemd \
nmap \
arp-scan \
nftables \
rsyslog \
chrony \
jq \
iputils-ping \
net-tools \
sshpas
```

```
echo "apt pleinement installé"
```

```
# Vérification que batctl est disponible
if ! command -v batctl >/dev/null 2>&1; then
    log "ERREUR : batctl introuvable après installation"
    step_fail
fi
```

```
log "batctl disponible : $(batctl -v)"
step_ok
echo "Etape 1 OK."
```

```
#####
# [2/16] Désactivation NetworkManager / WPA
#####
echo "[2/16] Désactivation services conflictuels..."
step_start "[2/16] Désactivation services conflictuels..." "2/16"
```

```
log "[ACTION] Désactivation NetworkManager"
systemctl disable --now NetworkManager >>"$LOGFILE" 2>&1 || true
log_systemd_status NetworkManager
```

```
log "[ACTION] Désactivation wpa_supplicant"
systemctl disable --now wpa_supplicant >>"$LOGFILE" 2>&1 || true
log_systemd_status wpa_supplicant
```

```
step_ok
echo "Etape 2 OK."
```

```
#####
# [3/16] Activation systemd-networkd
#####
echo "[3/16] Activation systemd-networkd..."
```

```
step_start "[3/16] Activation systemd-networkd..." "3/16"
```

```
systemctl enable systemd-networkd >>"$LOGFILE" 2>&1
```

```
systemctl start systemd-networkd >>"$LOGFILE" 2>&1
```

```
log_systemd_status systemd-networkd
```

```
step_ok
```

```
echo "Etape 3 OK."
```

```
#####
```

```
# [4/16] Synchronisation Date & Heure – Mesh Chrony
```

```
#####
```

```
echo "[4/16] Configuration NTP Mesh (chrony)..."
```

```
step_start "[4/16] Configuration NTP Mesh (chrony)..." "4/16"
```

```
systemctl disable --now systemd-timesyncd >/dev/null 2>&1 || true
```

```
if [ "$BOARD_ID" -eq 1 ]; then
```

```
    log "NTP Mesh : configuration NOEUD MAÎTRE (10.0.0.1)"
```

```
    cat > /etc/chrony/chrony.conf <<EOF
```

```
# === Chrony MASTER – Mesh 10.0.0.0/24 ===
```

```
# Ce noeud est la référence de temps locale
```

```
local stratum 10
```

```
# Autoriser les clients du mesh
```

```
allow 10.0.0.0/24
```

```
# Corriger l'heure brutalement au boot si nécessaire
```

```
makestep 1.0 3
```

```
# Logs
```

```
logdir /var/log/chrony
```

```
EOF
```

```
else
```

```
    log "NTP Mesh : configuration NOEUD CLIENT (sync sur 10.0.0.1)"
```

```
    cat > /etc/chrony/chrony.conf <<EOF
```

```
# === Chrony CLIENT – Mesh ===
```

```
# Serveur NTP unique
```

```
server 10.0.0.1 iburst
```

```
# Correction rapide si dérive importante
```

```
makestep 1.0 3
```

```

# Logs
logdir /var/log/chrony
EOF
fi

systemctl enable chrony >>"$LOGFILE" 2>&1
systemctl restart chrony >>"$LOGFILE" 2>&1
log_systemd_status chrony

cat > /usr/local/bin/mesh-ntp-sync.sh <<'EOF'
#!/bin/bash

LOG="/var/log/mesh-ntp.log"
echo "=== Mesh NTP sync $(date) ===" >> "$LOG"

# Attente IP mesh sur bat0
for i in {1..30}; do
    if ip addr show bat0 | grep -q "10.0.0."; then
        break
    fi
    sleep 1
done

# Resynchronisation forcée
chronyc -a makestep >> "$LOG" 2>&1
chronyc tracking >> "$LOG" 2>&1
EOF

chmod +x /usr/local/bin/mesh-ntp-sync.sh

cat > /etc/systemd/system/mesh-ntp-sync.service <<EOF
[Unit]
Description=Mesh NTP Resynchronisation (chrony)
After=mesh-batman.service
Wants=mesh-batman.service

[Service]
Type=oneshot
ExecStart=/usr/local/bin/mesh-ntp-sync.sh

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload >>"$LOGFILE" 2>&1
systemctl enable mesh-ntp-sync.service >>"$LOGFILE" 2>&1

step_ok

```

```
echo "Etape 4 OK."
```

```
#####
```

```
# [5/16] Chargement du module batman-adv
```

```
#####
```

```
echo "[5/16] Configuration module batman-adv..."
```

```
step_start "[5/16] Configuration module batman-adv..." "5/16"
```

```
# Chargement immédiat
```

```
if ! modprobe batman-adv >>"$LOGFILE" 2>&1; then
```

```
    log "ERREUR : impossible de charger le module batman-adv"
```

```
    log "Vérifiez que le module est bien présent : modinfo batman-adv"
```

```
    step_fail
```

```
fi
```

```
# Persistance au boot
```

```
echo "batman-adv" > /etc/modules-load.d/batman-adv.conf
```

```
log "Module batman-adv chargé et persistant"
```

```
# Vérification
```

```
if lsmod | grep -q "^batman_adv"; then
```

```
    log "Module batman_adv actif : $(lsmod | grep batman_adv)"
```

```
else
```

```
    log "AVERTISSEMENT : module batman_adv non visible dans lsmod"
```

```
fi
```

```
# Paramètres kernel pour le routage mesh
```

```
cat > /etc/sysctl.d/99-mesh-routing.conf <<EOF
```

```
net.ipv4.ip_forward=1
```

```
net.ipv6.conf.all.forwarding=1
```

```
net.ipv4.conf.all.send_redirects=0
```

```
net.ipv4.conf.default.send_redirects=0
```

```
net.ipv4.conf.all.rp_filter=0
```

```
net.ipv4.conf.default.rp_filter=0
```

```
net.ipv4.conf.all.accept_source_route=1
```

```
net.ipv4.conf.default.accept_source_route=1
```

```
# ICMP broadcast pour découverte des voisins
```

```
net.ipv4.icmp_echo_ignore_broadcasts=0
```

```
EOF
```

```
sysctl --system >>"$LOGFILE" 2>&1
```

```
step_ok
```

```
echo "Etape 5 OK."
```

```
#####
# [6/16] Exclure wlan0 et bat0 de systemd-networkd
#####
echo "[6/16] Exclusion de wlan0 et bat0 de systemd-networkd..."
step_start "[6/16] Exclusion wlan0/bat0 de systemd-networkd..." "6/16"

rm -f /etc/systemd/network/*wlan0*.network
rm -f /etc/systemd/network/*bat0*.network
rm -f /etc/systemd/network/99-ignore-wlan0.link
rm -f /etc/systemd/network/99-ignore-bat0.link

cat > /etc/systemd/network/99-ignore-wlan0.link <<EOF
[Match]
OriginalName=wlan0

[Link]
Unmanaged=yes
LinkLocalAddressing=no
DHCP=no
IPv6AcceptRA=no
EOF

cat > /etc/systemd/network/99-ignore-bat0.link <<EOF
[Match]
OriginalName=bat0

[Link]
Unmanaged=yes
LinkLocalAddressing=no
DHCP=no
IPv6AcceptRA=no
EOF

log "[ACTION] Restart systemd-networkd"
systemctl restart systemd-networkd >>"$LOGFILE" 2>&1
log_systemd_status systemd-networkd

step_ok
echo "Etape 6 OK."

#####
# [7/16] Désactivation IPv4 link-local (évite les 169.254.x.x)
#####
echo "[7/16] Désactivation IPv4 link-local..."
step_start "[7/16] Désactivation IPv4 link-local..." "7/16"

cat > /etc/sysctl.d/99-mesh-no-ipv4ll.conf <<EOF
```

```
net.ipv4.conf.all.autoconf=0
net.ipv4.conf.default.autoconf=0
net.ipv4.conf.wlan0.autoconf=0
net.ipv4.conf.all.accept_local=0
net.ipv4.conf.default.accept_local=0
EOF
```

```
sysctl --system >>"$LOGFILE" 2>&1
```

```
step_ok
echo "Etape 7 OK."
```

```
#####
# [8/16] Script principal BATMAN-adv
#####
echo "[8/16] Script de configuration BATMAN-adv..."
step_start "[8/16] Script BATMAN-adv..." "8/16"
```

```
cat > /usr/local/bin/setup-batman.sh <<'BATSCRIPT'
#!/bin/bash
# setup-batman.sh — mesh BATMAN-adv pour BCM43438 (Pi Zero W)
#
# SÉQUENCE FINALE VALIDÉE :
# 1. rfkill reset
# 2. iw set type ibss
# 3. iw ibss join + attente association stable (10s)
# 4. batctl if add (sur wlan0 DÉJÀ en IBSS stable)
# 5. bat0 up + IP
#
# Le ibss join DOIT être fait AVANT batctl if add, sinon batman-adv
# perd wlan0 quand le driver fait un reset interne lors du join.
```

```
LOG="/var/log/mesh-batman-setup.log"
echo "=== Setup BATMAN-adv $(date) ===" >> "$LOG"
```

```
BOARD_ID=$(cat /etc/mesh-id 2>/dev/null)
if [ -z "$BOARD_ID" ]; then
    echo "ERREUR : /etc/mesh-id introuvable" >> "$LOG"
    exit 1
fi
```

```
IP="10.0.0.$BOARD_ID"
ESSID="mesh-test"
CHANNEL_FREQ="2437"
```

```
echo "BOARD_ID=$BOARD_ID IP=$IP" >> "$LOG"
```

— ETAPE 1 : reset firmware

```
echo "[1/5] Reset firmware BCM43438..." >> "$LOG"
```

```
batctl if del wlan0 2>>"$LOG" || true  
sleep 1
```

```
ip link set wlan0 down 2>>"$LOG" || true  
sleep 1
```

```
rkill block wifi 2>>"$LOG"  
sleep 2  
rkill unblock wifi 2>>"$LOG"  
sleep 3
```

```
for i in $(seq 1 10); do  
    if ip link show wlan0 >/dev/null 2>&1; then  
        echo "    wlan0 OK après ${i}s" >> "$LOG"  
        break  
    fi  
    sleep 1  
done
```

— ETAPE 2 : mode IBSS

```
echo "[2/5] Mode IBSS..." >> "$LOG"
```

```
ip link set wlan0 down 2>>"$LOG"  
sleep 1
```

```
iw dev wlan0 set type ibss 2>>"$LOG"  
echo "    RC set type ibss : $" >> "$LOG"
```

```
ip link set wlan0 up 2>>"$LOG"  
sleep 1
```

```
# Augmenter le MTU pour batman-adv (requis 1532, défaut 1500)  
echo "    MTU wlan0 → 1532 (requis batman-adv)..." >> "$LOG"  
ip link set wlan0 mtu 1532 2>>"$LOG"  
echo "    RC set mtu : $" >> "$LOG"
```

```
sleep 1
```

— ETAPE 3 : ibss join AVANT batctl if add

```
# CRITIQUE : wlan0 doit être stable en IBSS avant que batman la prenne  
echo "[3/5] ibss join (AVANT batctl if add)..." >> "$LOG"
```

```

JOINED=false
for attempt in 1 2 3; do
    echo "  Tentative $attempt..." >> "$LOG"
    iw dev wlan0 ibss join "$ESSID" "$CHANNEL_FREQ" 2>>"$LOG"
    echo "  RC ibss join : $" >> "$LOG"

    # Attendre association stable (10s max)
    for wait in $(seq 1 10); do
        sleep 1
        LINK=$(iw dev wlan0 link 2>/dev/null)
        if echo "$LINK" | grep -qi "Joined\|Connected"; then
            BSSID=$(echo "$LINK" | awk '/Joined IBSS/{print $3}')
            echo "  Association OK après ${wait}s — BSSID : $BSSID" >> "$LOG"
            JOINED=true
            break 2
        fi
    done

    echo "  Pas associé, nouvelle tentative..." >> "$LOG"
    iw dev wlan0 ibss leave 2>>"$LOG" || true
    sleep 2
done

if [ "$JOINED" = false ]; then
    echo "  ERREUR : association IBSS échouée après 3 tentatives" >> "$LOG"
    exit 1
fi

# Laisser l'association se stabiliser AVANT de passer à batman
echo "  Stabilisation association..." >> "$LOG"
sleep 5

echo "  État wlan0 avant batctl if add : " >> "$LOG"
iw dev wlan0 info >> "$LOG" 2>&1
iw dev wlan0 link >> "$LOG" 2>&1

# — ETAPE 4 : batctl if add (sur wlan0 DÉJÀ stable)


---


echo "[4/5] batctl if add wlan0 (wlan0 déjà en IBSS stable)..." >> "$LOG"

batctl if add wlan0 2>>"$LOG"
echo "  RC batctl if add : $" >> "$LOG"
sleep 2

# Vérification critique
BATMAN_IF=$(batctl if 2>/dev/null)
echo "  Interfaces batman : ${BATMAN_IF:-VIDE}" >> "$LOG"

```



```
if [ -z "$BATMAN_IF" ]; then
    echo "  ERREUR : batman n'a pas pris wlan0" >> "$LOG"
    exit 1
fi
```

```
MASTER=$(ip link show wlan0 2>/dev/null | grep -o "master [^ ]*" | awk '{print $2}')
echo "  wlan0 master : ${MASTER:-aucun}" >> "$LOG"
```

```
# Attendre que batman stabilise wlan0 (important)
sleep 5
```

```
# Vérifier que batman n'a pas perdu wlan0
BATMAN_IF_CHECK=$(batctl if 2>/dev/null)
if [ -z "$BATMAN_IF_CHECK" ]; then
    echo "  ERREUR : batman a perdu wlan0 après 5s (voir dmesg)" >> "$LOG"
    dmesg | grep batman_adv | tail -10 >> "$LOG"
    exit 1
fi
```

```
echo "  Batman a gardé wlan0 — OK" >> "$LOG"
```

```
# Flush IP wlan0 pour éviter 169.254.x.x
ip addr flush dev wlan0 2>>"$LOG" || true
```

```
# — ETAPE 5 : bat0 up + IP
```

```
echo "[5/5] Configuration bat0 avec IP ${IP}/24..." >> "$LOG"
```

```
ip link set bat0 up 2>>"$LOG"
sleep 1
ip addr flush dev bat0 2>>"$LOG"
ip addr add "${IP}/24" dev bat0 2>>"$LOG"
echo "  RC ip addr add : $?" >> "$LOG"
```

```
# — Résumé
```

```
echo "=== Résumé final ===" >> "$LOG"
echo "--- iw dev ---" >> "$LOG"
iw dev >> "$LOG" 2>&1
echo "--- wlan0 link ---" >> "$LOG"
ip link show wlan0 >> "$LOG" 2>&1
echo "--- bat0 ---" >> "$LOG"
ip addr show bat0 >> "$LOG" 2>&1
echo "--- batman interfaces ---" >> "$LOG"
batctl if >> "$LOG" 2>&1
echo "--- batman neighbors (attendre 10s) ---" >> "$LOG"
sleep 10
```

```
batctl neighbors >> "$LOG" 2>&1
echo "=== Fin setup $(date) ===" >> "$LOG"
```

```
exit 0
BATSCRIPT
```

```
chmod +x /usr/local/bin/setup-batman.sh
log "Script BATMAN-adv prêt"
```

```
step_ok
echo "Etape 8 OK."
```

```
#####
# [9/16] Service systemd – BATMAN-adv
#####
echo "[9/16] Service systemd BATMAN-adv..."
step_start "[9/16] Service systemd BATMAN-adv..." "9/16"
```

```
cat > /etc/systemd/system/mesh-batman.service <<EOF
[Unit]
Description=Mesh BATMAN-adv Network (wlan0 + bat0)
After=sys-subsystem-net-devices-wlan0.device
Requires=sys-subsystem-net-devices-wlan0.device
```

```
[Service]
Type=oneshot
ExecStart=/usr/local/bin/setup-batman.sh
RemainAfterExit=yes
Restart=on-failure
RestartSec=5
```

```
[Install]
WantedBy=multi-user.target
EOF
```

```
log "[ACTION] systemd daemon-reload"
systemctl daemon-reload >>"$LOGFILE" 2>&1
```

```
log "[ACTION] Activation mesh-batman.service"
systemctl enable mesh-batman.service >>"$LOGFILE" 2>&1
log_systemd_status mesh-batman.service
```

```
step_ok
echo "Etape 9 OK."
```

```
#####
# [10/16] Ping broadcast périodique
#####
```

```
echo "[10/16] Ping broadcast mesh..."
step_start "[10/16] Ping broadcast mesh..." "10/16"

cat > /usr/local/bin/mesh-broadcast-ping.sh <<'EOF'
#!/bin/bash

INTERFACE="bat0"
BROADCAST_IP="10.0.0.255"
INTERVAL=10

echo "=== Mesh broadcast ping started $(date) ==="
echo "Interface : $INTERFACE"
echo "Broadcast : $BROADCAST_IP"
echo "Intervalle : ${INTERVAL}s"

# Attente bat0 UP
while true; do
    if ip link show "$INTERFACE" 2>/dev/null | grep -q "UP"; then
        break
    fi
    sleep 1
done

sysctl -w net.ipv4.icmp_echo_ignore_broadcasts=0 >/dev/null

while true; do
    ping -b -c 1 -W 1 "$BROADCAST_IP" >/dev/null 2>&1
    sleep "$INTERVAL"
done
EOF

chmod +x /usr/local/bin/mesh-broadcast-ping.sh

cat > /etc/systemd/system/mesh-broadcast-ping.service <<EOF
[Unit]
Description=Mesh Broadcast Ping (10.0.0.255)
After=mesh-batman.service
Wants=mesh-batman.service

[Service]
Type=simple
ExecStart=/usr/local/bin/mesh-broadcast-ping.sh
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
```

```
systemctl daemon-reload
systemctl enable mesh-broadcast-ping.service >>"$LOGFILE" 2>&1
systemctl start mesh-broadcast-ping.service >>"$LOGFILE" 2>&1
```

```
step_ok
echo "Etape 10 OK."
```

```
#####
# [11/16] Script de vérification BATMAN-adv post-boot
#####
echo "[11/16] Script vérification BATMAN-adv post-boot..."
step_start "[11/16] Script vérification BATMAN-adv post-boot..." "11/16"
```

```
cat > /usr/local/bin/mesh-check-batman.sh <<'EOF'
#!/bin/bash
```

```
LOG="/var/log/mesh-batman-check.log"
echo "=== Vérification BATMAN-adv $(date) ===" >> "$LOG"
```

```
# Attente bat0 UP
for i in {1..30}; do
    if ip link show bat0 2>/dev/null | grep -q "UP"; then
        break
    fi
    sleep 1
done
```

```
# Attente IP configurée sur bat0
for i in {1..30}; do
    if ip addr show bat0 | grep -q "10.0.0."; then
        break
    fi
    sleep 1
done
```

```
sleep 5
```

```
# Vérification module
if ! lsmod | grep -q "^batman_adv"; then
    echo "ERREUR : module batman_adv non chargé, tentative..." >> "$LOG"
    modprobe batman-adv
    sleep 2
fi
```

```
# Vérification service actif
if ! systemctl is-active --quiet mesh-batman.service; then
    echo "ERREUR: mesh-batman.service inactif, redémarrage..." >> "$LOG"
```

```

systemctl restart mesh-batman.service
sleep 10
fi

# Affichage voisins BATMAN-adv
echo "--- Voisins BATMAN-adv ---" >> "$LOG"
batctl neighbors >> "$LOG" 2>&1

# Affichage table de routage
echo "--- Table de routage BATMAN-adv ---" >> "$LOG"
batctl routing_algo >> "$LOG" 2>&1
batctl tg >> "$LOG" 2>&1

# Vérification forwarding
if [ "$(cat /proc/sys/net/ipv4/ip_forward)" != "1" ]; then
    echo "ERREUR: Forwarding désactivé, correction..." >> "$LOG"
    sysctl -w net.ipv4.ip_forward=1
fi

echo "=== Fin vérification $(date) ===" >> "$LOG"
EOF

chmod +x /usr/local/bin/mesh-check-batman.sh

cat > /etc/systemd/system/mesh-check-batman.service <<EOF
[Unit]
Description=Mesh BATMAN-adv Post-Boot Check
After=mesh-batman.service
Wants=mesh-batman.service

[Service]
Type=oneshot
ExecStart=/usr/local/bin/mesh-check-batman.sh
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload >>"$LOGFILE" 2>&1
systemctl enable mesh-check-batman.service >>"$LOGFILE" 2>&1

step_ok
echo "Etape 11 OK."

#####
# [12/16] Logs ICMP JSON temps réel
#####

```

```

echo "[12/16] Logs ICMP JSON temps réel..."
step_start "[12/16] Logs ICMP JSON temps réel..." "12/16"

cat > /usr/local/bin/mesh-icmp-json.sh <<'EOF'
#!/bin/bash

LOG_JSON="/var/log/mesh-icmp.json"
INTERFACE="bat0"
PING_INTERVAL=1
DURATION=600

START_EPOCH="$1"

if [ -z "$START_EPOCH" ]; then
    echo "Usage: mesh-icmp-json.sh <start_epoch>"
    exit 1
fi

# Attente synchronisée précise
while true; do
    NOW=$(date +%s)
    [ "$NOW" -ge "$START_EPOCH" ] && break
    sleep 0.02
done

REAL_START_EPOCH=$(date +%s)
REAL_START_HUMAN=$(date '+%Y-%m-%d %H:%M:%S')

BOARD_ID=$(cat /etc/mesh-id 2>/dev/null)
NODE_IP="10.0.0.$BOARD_ID"

TMP_FILE="/tmp/mesh-icmp-full.tmp"

echo "{" > "$TMP_FILE"
echo "  \"node\": \"$NODE_IP\"," >> "$TMP_FILE"
echo "  \"scheduled_start_epoch\": $START_EPOCH," >> "$TMP_FILE"
echo "  \"real_start_epoch\": $REAL_START_EPOCH," >> "$TMP_FILE"
echo "  \"real_start_time\": \"$REAL_START_HUMAN\"," >> "$TMP_FILE"
echo "  \"iterations\": [" >> "$TMP_FILE"

ITER=0
END_TIME=$((REAL_START_EPOCH + DURATION))

while [ "$(date +%s)" -lt "$END_TIME" ]; do

    [ $ITER -gt 0 ] && echo "," >> "$TMP_FILE"

    echo "    {" >> "$TMP_FILE"

```

```

echo "    \"timestamp\": \"$(date '+%Y-%m-%d %H:%M:%S')\", \">> \"$TMP_FILE"
echo "    \"epoch\": $(date +%s),\">> \"$TMP_FILE"
echo "    \"batman_neighbors\": [\">> \"$TMP_FILE"

# Récupération des voisins via batctl
BAT_NEIGH=$(batctl neighbors 2>/dev/null | tail -n +2 | awk '{print $1}' | grep -v
'^$')

# Récupération des voisins IP depuis le cache ARP sur bat0
ACTIVE_NEIGH=$(ip neigh show dev "$INTERFACE" 2>/dev/null | \
    awk '/REACHABLE|STALE|DELAY/ {print $1}' | \
    grep '^10\.\0\.\0\.')

FIRST=true

for IP in $ACTIVE_NEIGH; do

    PING=$(ping -I "$INTERFACE" -c 1 -W 1 "$IP" 2>/dev/null)

    if [ $? -eq 0 ]; then
        TTL=$(echo "$PING" | grep 'ttl=' | sed -E 's/. *ttl=([0-9]+).*/\1/')
        LAT=$(echo "$PING" | grep 'time=' | sed -E 's/. *time=([0-9\.\.]+).*/\1/')

        ROUTE_INFO=$(ip route get "$IP" dev "$INTERFACE" 2>/dev/null)
        NEXT_HOP=$(echo "$ROUTE_INFO" | awk '/via/ {print $3}')
        [ -z "$NEXT_HOP" ] && NEXT_HOP="DIRECT"

        if [ "$FIRST" = true ]; then
            FIRST=false
        else
            echo ",">> "$TMP_FILE"
        fi

        echo "    {">> "$TMP_FILE"
        echo "        \"IP\": \"$IP\", \">> \"$TMP_FILE"
        echo "        \"TTL\": $TTL, \">> \"$TMP_FILE"
        echo "        \"LATENCY_MS\": $LAT, \">> \"$TMP_FILE"
        echo "        \"NEXT_HOP\": \"$NEXT_HOP\" \">> \"$TMP_FILE"
        echo -n "    }\">> "$TMP_FILE"

    fi
done

echo "">> "$TMP_FILE"
echo "    ]\">> "$TMP_FILE"
echo -n "    }\">> "$TMP_FILE"

ITER=$((ITER+1))
sleep $PING_INTERVAL

```

done

```
echo "" >> "$TMP_FILE"
echo " ]" >> "$TMP_FILE"
echo "]" >> "$TMP_FILE"
```

```
mv "$TMP_FILE" "$LOG_JSON"
exit 0
EOF
```

```
chmod +x /usr/local/bin/mesh-icmp-json.sh
```

```
cat > /etc/systemd/system/mesh-icmp-json.service <<EOF
[Unit]
Description=Mesh ICMP JSON Logging (600s)
After=mesh-batman.service
Wants=mesh-batman.service
```

```
[Service]
Type=simple
ExecStart=/usr/local/bin/mesh-icmp-json.sh
RemainAfterExit=no
```

```
[Install]
WantedBy=multi-user.target
EOF
```

```
systemctl daemon-reload
systemctl disable mesh-icmp-json.service
```

```
log "Service ICMP JSON prêt (manuel uniquement)"
```

```
step_ok
echo "Etape 12 OK."
```

```
#####
# [13/16] Script maître – Déclenchement ICMP global
#####
echo "[13/16] Préparation déclenchement ICMP global (manuel)..."
step_start "[13/16] Déclenchement ICMP global manuel" "13/16"
```

```
if [ "$BOARD_ID" -eq 1 ]; then
```

```
cat > /usr/local/bin/mesh-run-icmp <<'EOF'
#!/bin/bash
```

```
USER="rpi"
PASS="rpi"
```



```

NETWORK="10.0.0.0/24"
DURATION=600
START_DELAY=30
LOG="/var/log/mesh-master-sync.log"
LOCAL_DIR="/home/pi/mesh-logs"

mkdir -p "$LOCAL_DIR"

echo "===== " | tee -a "$LOG"
echo "Déclenchement synchronisé $(date)" | tee -a "$LOG"
echo "===== " | tee -a "$LOG"

START_EPOCH=$(( $(date +%s) + START_DELAY ))
HUMAN_START=$(date -d @$START_EPOCH '+%Y-%m-%d %H:%M:%S')

echo "Départ programmé à $HUMAN_START (epoch $START_EPOCH)" | tee -a
"$LOG"

NODES=$(nmap -sn $NETWORK | awk '/Nmap scan report/{print $5}' | grep 10.0.0.)

for IP in $NODES; do
    echo "Envoi ordre à $IP" | tee -a "$LOG"

    sshpass -p "$PASS" ssh -o StrictHostKeyChecking=no -o ConnectTimeout=5 \
        $USER@$IP \
        "sudo /usr/local/bin/mesh-icmp-json.sh $START_EPOCH > /dev/null 2>&1 &" &
done

wait

echo "Tous les ordres envoyés" | tee -a "$LOG"
echo "Attente fin acquisition..." | tee -a "$LOG"

sleep $((START_DELAY + DURATION + 5))

for IP in $NODES; do
    ID=$(echo $IP | awk -F. '{print $4}')
    DEST="$LOCAL_DIR/log.$ID.json"

    sshpass -p "$PASS" scp -o StrictHostKeyChecking=no \
        $USER@$IP:/var/log/mesh-icmp.json \
        "$DEST" > /dev/null 2>&1

    if [ -f "$DEST" ]; then
        echo "Récupéré $DEST" | tee -a "$LOG"
    else
        echo "Echec récupération $IP" | tee -a "$LOG"
    fi

```

```

done

echo "Synchronisation terminée $(date)" | tee -a "$LOG"
EOF

chmod +x /usr/local/bin/mesh-run-icmp

echo "alias mesh-run-icmp='sudo /usr/local/bin/mesh-run-icmp'" >> /home/pi/.bashrc

log "Commande disponible : sudo mesh-run-icmp"

fi

step_ok
echo "Etape 13 OK."

#####
# [14/16] Reset IP bat0 après boot
#####
echo "[14/16] Reset IP bat0 après boot..."
step_start "[14/16] Reset IP bat0 après boot..." "14/16"

cat > /usr/local/bin/mesh-reset-ip.sh <<'EOF'
#!/bin/bash

LOG="/var/log/mesh-reset-ip.log"
echo "=== Mesh reset IP $(date) ===" >> "$LOG"

sleep 15

if [ ! -f /etc/mesh-id ]; then
    echo "Erreur : /etc/mesh-id introuvable" >> "$LOG"
    exit 1
fi
BOARD_ID=$(cat /etc/mesh-id)
IP="10.0.0.$BOARD_ID"
echo "BOARD_ID=$BOARD_ID, IP=$IP" >> "$LOG"

# Vérification bat0
if ! ip link show bat0 >/dev/null 2>&1; then
    echo "Erreur : bat0 introuvable, relance du service mesh-batman..." >> "$LOG"
    systemctl restart mesh-batman.service
    sleep 10
fi

# Nettoyage wlan0 : flush IP uniquement, PAS de DOWN
# batman-adv a besoin que wlan0 reste UP comme lien physique radio
echo "Flush IP wlan0 (reste UP pour batman-adv)..." >> "$LOG"

```

```

ip addr flush dev wlan0 2>>"$LOG" || true
sysctl -w net.ipv4.conf.wlan0.autoconf=0 >> "$LOG" 2>&1

echo "Flush IP bat0..." >> "$LOG"
ip addr flush dev bat0 2>>"$LOG" || true

sleep 2

echo "Remise IP ${IP}/24 sur bat0..." >> "$LOG"
ip link set bat0 up 2>>"$LOG"
ip addr add "${IP}/24" dev bat0 2>>"$LOG"

echo "=== Etat final ===" >> "$LOG"
ip addr show wlan0 >> "$LOG" 2>&1
ip addr show bat0 >> "$LOG" 2>&1
batctl if >> "$LOG" 2>&1
batctl neighbors >> "$LOG" 2>&1

echo "=== Fin Mesh reset IP $(date) ===" >> "$LOG"
EOF

chmod +x /usr/local/bin/mesh-reset-ip.sh

cat > /etc/systemd/system/mesh-reset-ip.service <<EOF
[Unit]
Description=Reset IP bat0 after boot
After=mesh-batman.service
Wants=mesh-batman.service

[Service]
Type=oneshot
ExecStart=/usr/local/bin/mesh-reset-ip.sh
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload >>"$LOGFILE" 2>&1
systemctl enable mesh-reset-ip.service >>"$LOGFILE" 2>&1

step_ok
echo "Etape 14 OK."

#####
# [15/16] Script de diagnostic mesh-info + alias
#####
echo "[15/16] Alias de diagnostic BATMAN-adv..."

```

```
cat > /usr/local/bin/mesh-info <<'MESHINFO'
#!/bin/bash
# mesh-info : diagnostic complet du mesh BATMAN-adv
```

```
board_id=$(cat /etc/mesh-id 2>/dev/null || echo "?")
```

```

"${BLU}" || _____ || "${NC}"
echo -e "${BLU}" ||   Mesh BATMAN-adv — Noeud $board_id   || "${NC}"
echo -e
"${BLU}" || _____ || "${NC}"
echo ""

```

```
echo -e "${YEL}— Interfaces reseau
_____${NC}"
```

```
for IF in wlan0 bat0; do
    STATE=$(cat /sys/class/net/$IF/operstate 2>/dev/null || echo "absent")
    ADDRS=$(ip -4 addr show $IF 2>/dev/null | awk '/inet /{print $2}' | tr '\n' ' ')
    LL=$(ip -4 addr show $IF 2>/dev/null | awk '/inet 169\./{print $2}' | tr '\n' ' ')

    [ "$STATE" = "up" ] && S="{GRN}UP${NC}" || S="{RED}$STATE${NC}"
    [ -n "$LL" ] && WARN="{RED} << WARN: 169.254 presente !${NC}" ||
    =""

    [ -z "$ADDRS" ] && ADDRS="—"

    printf " %-8s %-28s " "$IF" "$ADDRS"
    echo -e "$S$WARN"
done
```

```
# Avertissement explicite wlan0
LL_WLAN=$(ip -4 addr show wlan0 2>/dev/null | awk '/169\./{print $2}')
if [ -n "$LL_WLAN" ]; then
    echo ""
    echo -e "  ${RED}[!] wlan0 a toujours : $LL_WLAN${NC}"
    echo -e "  ${YEL}  Corriger avec : mesh-wlan0-clean${NC}"
fi
```

fi

```
ADDRS6=$(ip -6 addr show bat0 2>/dev/null | awk '/inet6 /{print $2}' | grep -v '^fe80' |
tr '\n' ' ')
[ -n "$ADDRS6" ] && echo -e " bat0 IPv6 : ${CYN}${ADDRS6}${NC}"
echo ""

# — Voisins BATMAN-adv avec IP et latence —————
echo -e "${YEL}— Voisins BATMAN-adv (MAC, IP, latence) ————${NC}"
printf " %-19s %-16s %-14s %s\n" "MAC voisin" "IP 10.0.0.x" "Latence moy." "Vu il
y a"
echo "
```

```
# Table ARP bat0 : MAC → IP (source la plus fiable)
declare -A ARP_MAC
while IFS= read -r line; do
    IP=$(echo "$line" | awk '{print $1}')
    MAC=$(echo "$line" | awk '{print $5}')
    [[ "$IP" =~ ^10\.\0\.\0\.\ ] && [ -n "$MAC" ] && ARP_MAC["$MAC"]="$IP"
done < <(ip neigh show dev "$IFACE" 2>/dev/null)
```

```
# Translation table batman : MAC → IP (fallback)
declare -A TG_MAC
while IFS= read -r line; do
    MAC=$(echo "$line" | awk '{print $1}')
    IP=$(echo "$line" | grep -oP '10\.\0\.\0\.[0-9]+' | head -1)
    [ -n "$MAC" ] && [ -n "$IP" ] && TG_MAC["$MAC"]="$IP"
done < <(batctl tg 2>/dev/null | tail -n +3)
```

```
HAS=false
while IFS= read -r line; do
    [[ -z "${line//[[:space:]]/}" ]] && continue
    [[ "$line" =~ ^[[:space:]]*IF ]] && continue
```

```
MAC=$(echo "$line" | awk '{print $2}')
SEEN=$(echo "$line" | awk '{print $3}')
[[ -z "$MAC" ]] && continue
```

```
TARGET="${ARP_MAC[$MAC]:-$TG_MAC[$MAC]:-}"
```

```
if [ -n "$TARGET" ]; then
    POUT=$(ping -I "$IFACE" -c 2 -W 1 -q "$TARGET" 2>/dev/null)
    LAT=$(echo "$POUT" | grep -oP 'rtt.+= \K[0-9.]+(?:/[0-9.]+/[0-9.]+)')
    [ -z "$LAT" ] && LAT=$(echo "$POUT" | grep -oP
'[0-9.]+\K[0-9.]+(?:/[0-9.]+/[0-9.]+)')
    if [ -n "$LAT" ]; then
        LAT_STR="${GRN}${LAT} ms${NC}"
```

```

        else
            LAT_STR="${RED}injoignable${NC}"
        fi
    else
        TARGET="— (ARP vide)"
        LAT_STR="${YEL}—${NC}"
    fi

    printf " %-19s %-16s " "$MAC" "$TARGET"
    echo -e "${LAT_STR}      ${SEEN}"
    HAS=true
done < <(batctl neighbors 2>/dev/null)

$HAS || echo -e " ${RED}Aucun voisin BATMAN-adv detecte${NC}"
echo ""

```

— Routes actives

```

echo -e "${YEL}— Routes actives (bat0)
                                     —${NC}"
ip route show dev bat0 2>/dev/null | sed 's/^/ /' || echo " —"
echo ""

echo -e "${CYN} Commandes disponibles :${NC}"
echo "  mesh-info      → ce resume"
echo "  batctl tg        → translation table complete"
echo "  batctl ping <MAC> → ping L2 vers un voisin"
echo "  mesh-wlan0-clean → supprimer 169.254.x.x sur wlan0"
echo "  mesh-log         → log setup en direct"
echo ""
MESHINFO

chmod +x /usr/local/bin/mesh-info

cat >> /etc/bash.bashrc <<'ALIASES'

# === Alias mesh BATMAN-adv ===
alias mesh-info='/usr/local/bin/mesh-info'
alias mesh-neigh='batctl neighbors'
alias mesh-tg='batctl tg'
alias mesh-ping='batctl ping'
alias mesh-status='systemctl status mesh-batman.service
mesh-broadcast-ping.service mesh-reset-ip.service'
alias mesh-log='tail -f /var/log/mesh-batman-setup.log'
alias mesh-wlan0-clean='ip addr flush dev wlan0 2>/dev/null; sysctl -w
net.ipv4.conf.wlan0.autoconf=0 >/dev/null; echo "wlan0 : IPs flushees (reste UP pour
batman-adv)'"
ALIASES

```

```
step_ok
echo "Etape 15 OK."
```

```
#####
```

```
# [16/16] Identité & logs
```

```
#####
```

```
echo "[16/16] Finalisation..."
```

```
step_start "[16/16] Finalisation..." "16/16"
```

```
echo "$BOARD_ID" > /etc/mesh-id
```

```
mkdir -p /var/log
```

```
touch /var/log/mesh-adhoc.log
```

```
step_ok
```

```
echo "=====
```

```
echo " INSTALLATION TERMINÉE"
```

```
echo " Node ID : $BOARD_ID"
```

```
echo " IP      : $IP (sur bat0)"
```

```
echo " ESSID   : $ESSID"
```

```
echo " Proto   : BATMAN-adv"
```

```
echo "=====
```

```
echo ""
```

```
echo " Commandes utiles post-boot :"
```

```
echo "  batctl neighbors  → voisins directs"
```

```
echo "  batctl tg         → table de routage"
```

```
echo "  batctl ping <IP MAC> → ping couche 2"
```

```
echo "  mesh-run-icmp     → lancer acquisition (noeud 1 seulement)"
```

```
echo ""
```

```
echo "Redémarrage dans 30 secondes..."
```

```
sleep 30
```

```
reboot
```